

C++&Qt Day03 hw2 보고서

작성자: Doyeon Park

작성일: 2026.09.07

목차

1. 과제 개요
2. UI 구성
3. 맵 구현
4. Dijkstra 알고리즘 구현
5. A* 알고리즘 구현
6. A* 와 Dijkstra 알고리즘 비교
7. 결과 및 느낀 점

1. 과제 개요

이번 과제에서는 A* 알고리즘과 Dijkstra 알고리즘을 이용한 최단 경로 탐색 프로그램을 Qt 로 구현해보았다.

사용자가 맵의 크기를 직접 조정할 수 있고, 맵 위에 장애물과 시작점, 도착점을 직접 배치할 수 있도록 구성하였다. 이후 A* 와 Dijkstra 알고리즘을 동일한 맵위에서 실행하여 두 알고리즘의 도착점탐색 과정을 비교할 수 있도록 만들었다.

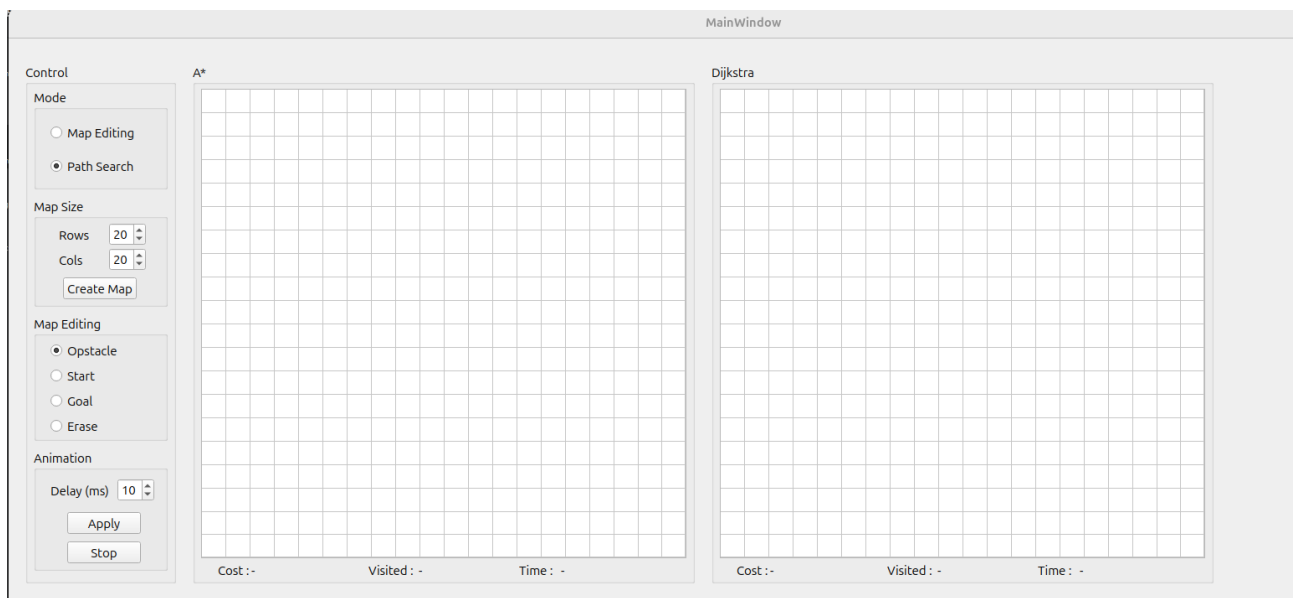
탐색 완료 후 최종 경로 비용, 탐색한 노드 수, 실행 시간을 화면에 표시하여 두 알고리즘을 비교할 수 있게 만들었다.

2. UI 구성

프로그램의 화면은 크게 Control 영역, A*영역, Dijkstra 영역으로 나누어 구성하였다.

먼저 Control 영역에서는 A*와 Dijkstra 맵을 edit 하거나 실행시키는 모드와 mapsize 를 조절할 수 있게 해주는 기능, 맵 편집시 출발점, 도착점, 장애물을 구분하여 설치할 수 있게 해주는 기능, 탐색 지연시간 및 탐색 시작, 멈춤 기능을 넣었다.

A*와 Dijkstra 영역은 control 영역에서 조정한 맵 크기를 띄우고 장애물, 시작점, 도착점을 표시하고 실행 과정 및 결과를 보여주는 영역이다.



3. 맵 구현

맵은 Qt 의 QTableWidgetItem 을 이용하여 구현하였다.Rows 와 Cols 에서 입력한 값을 이용하여 테이블의 행과 열 개수를 변경하였으며, 모든 셀이 현재 테이블 크기에 맞게 자동으로 조절되도록 하였다.

초기에는 20×20 크기에서는 정상적으로 표시되었지만 이 크기를 넘어서면 일부 셀이 화면 밖으로 넘어가 스크롤바가 생기는 문제가 있었다.

이를 해결하기 위해 각 셀의 최소 크기를 작게 설정하고 QHeaderView::Stretch 를 사용하여 테이블 크기에 맞게 셀 크기가 자동으로 줄어들도록 수정하였다. 그 결과 맵 크기를 크게 변경해도 전체 맵을 한 화면에서 확인할 수 있었다.

프로그램 내부에서는 맵의 상태를 2 차원 vector 에 저장하였다.

- 0: 빈칸
- 1: 장애물
- 2: 시작점
- 3: 도착점

따라서 QTableWidgetItem 은 화면에 맵을 보여주는 역할을 하고, 실제 알고리즘에서는 2 차원 vector 의 값을 이용하여 이동 가능 여부를 판단하도록 구성하였다.

4. Dijkstra 알고리즘 구현

Dijkstra 알고리즘에서는 시작점부터 현재 노드까지의 실제 누적 비용을 기준으로 탐색하였다.

현재 위치에서 상하좌우 네 방향을 확인하고, 맵의 범위를 벗어나거나 장애물인 경우에는 탐색하지 않도록 하였다.

각 칸으로 이동하는 비용은 모두 1로 설정하였다.

Dijkstra는 목적지가 어느 방향에 있는지 고려하지 않기 때문에 시작점을 중심으로 주변 영역을 넓게 탐색하는 모습을 확인할 수 있었다.

5. A* 알고리즘 구현

A* 알고리즘은 Dijkstra와 달리 실제 이동 비용뿐만 아니라 목적지까지의 예상 비용을 함께 사용하였다.

A*의 우선순위는 다음 식을 이용하였다.

$$f(n) = g(n) + h(n)$$

$g(n)$ 은 시작점에서 현재 노드까지 이동한 실제 비용이며, $h(n)$ 은 현재 위치에서 목적지까지의 예상 비용이다.

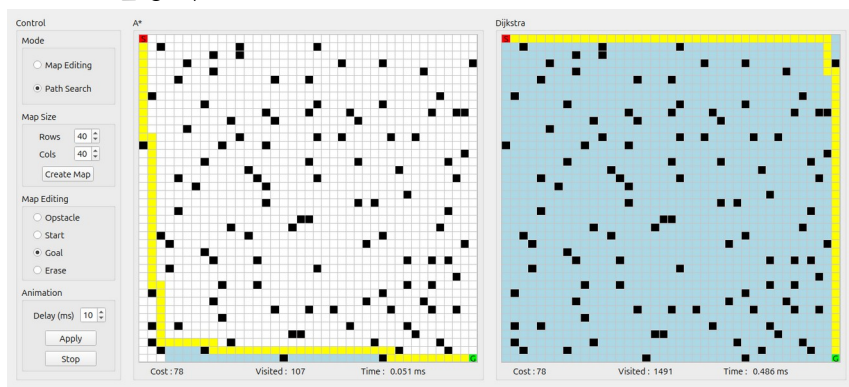
이번 구현에서는 2 차원 평면에서 작동하기 때문에 $h(n)$ 으로 맨허튼거리를 사용하였다.

$$h(n) = |\text{현재 행} - \text{목적지 행}| + |\text{현재 열} - \text{목적지 열}|$$

6. A* 와 Dijkstra 알고리즘 비교

실행 결과는 다음과 같다.

비교 항목	A*	Dijkstra
최종 경로 비용	78	78
탐색 노드 수	107	1491
실행 시간	0.051 ms	0.486 ms



각각의 결과를 비교해보면 탐색 노드 수에서 차이가 발생 한 것을 알 수 있다.

Dijkstra 는 시작점을 중심으로 넓은 영역을 탐색하여 1491 개의 노드를 방문하였지만, A*는 목적지까지의 예상 거리인 $h(n)$ 을 이용하여 107 개의 노드만 탐색하였다.

실행시간 또한 A* 가 더 짧게 측정되었다.

7. 결과 및 느낀 점

이번 과제를 통해 같은 최단 경로 탐색 문제라도 어떤 기준으로 다음 노드를 선택하는지에 따라 탐색 과정과 성능이 크게 달라질 수 있다는 점을 확인하였다. 또한 Dijkstra 는 현재까지의 실제 비용만을 기준으로 탐색하지만, A*는 실제 비용과 목적지까지의 예상 비용을 함께 이용한다는 차이를 프로그램의 탐색 과정을 직접 돌려보며 실감할 수 있었던 것 같다.